

DEVELOPMENT OF TEST SOLUTION FOR 3D SOC

A midterm capstone project report submitted in partial fulfilment of the requirement for the
award of the degree of

Bachelor of Engineering

in

ECE And ENC

Submitted By

Gautam Garg (Roll No. : 102306166)

Pulkita (Roll No. : 102315117)

Astha (Roll No. : 102315261)

Manya Thakur (Roll No. : 102315111)

Adish Garg (Roll No. : 102306071)

(Group No – 66)

Under Supervision of

Dr. Harpreet Vohra (Associate Professor, ECED)

Dr. Sumit Bansal (Assistant Professor, CSED)

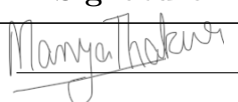


Department of Electronics and Communication Engineering
THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY, PATIALA,
PUNJAB
May 2026

DECLARATION

We hereby declare that the capstone project group report title “**Development of Test Solution for 3D SOC**” is an authentic record of our own work carried out at “Thapar Institute of Engineering and Technology, Patiala” as a Capstone Project in the sixth semester of B.E. (Electronics & Communication Engineering), under the guidance of Dr. Harpreet Vohra and Dr. Sumit Bansal, during January to May 2026.

Date: _____

Student Name	Roll No.	Signature
Manya Thakur	102315111	
Pulkita	102315117	_____
Astha	102315261	_____
Gautam Garg	102306166	_____
Adish Garg	102306071	_____

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our mentors, Dr. Harpreet Vohra and Dr. Sumit Bansal, for their continuous guidance, encouragement, and invaluable feedback throughout this Capstone Project. Their expertise in the domain of Electronics and Communication Engineering and Computer Science Engineering respectively heavily contributed to our understanding of 3D SoC architectures and optimization techniques.

We also extend our gratitude to the Department of Electronics and Communication Engineering, Thapar Institute of Engineering & Technology, for providing the necessary resources and infrastructure to carry out our hardware and software implementations.

Date: _____

Roll No.	Name
102315111	Manya Thakur
102315117	Pulkita
102315261	Astha
102306166	Gautam Garg
102306071	Adish Garg

ABSTRACT

The increasing complexity of three-dimensional System-on-Chip (3D SoC) architectures has created significant challenges in the testing process. The presence of Through-Silicon Vias (TSVs), high power use during testing, thermal hotspots, and routing issues often lead to longer Total Test Application Time (TAT) and lower testing efficiency. Therefore, creating an optimized test scheduling framework that reduces test time while meeting system limits has become a key research issue. Several test scheduling methods have been proposed in the literature, including Greedy scheduling and various optimization algorithms. While these methods effectively cut down test time, many existing solutions mainly focus on scheduling efficiency. They provide limited attention to thermal limits, TSV-related issues, and scalability for practical hardware use. This calls for a more thorough and constraint-aware scheduling method. In this work, we propose an optimized test scheduling framework for 3D SoC testing using metaheuristic algorithms like Modified Ant Colony Optimization (MACO), Bat Algorithm, Firefly Algorithm, and Artificial Bee Colony (ABC). The main goal is to minimize the Total Test Application Time while ensuring compliance with peak power, Test Access Mechanism (TAM) width, and TSV limits. The proposed solution uses a three-phase iterative method. In the first phase, we develop a 2D SoC scheduling framework that focuses on core-level test scheduling within power and TAM limits. In the second phase, we extend the framework to 3D SoC architectures by adding TSV-aware and thermal-aware scheduling strategies. In the third phase, we plan a hardware-assisted validation framework using Raspberry Pi 5 to test the practical feasibility of the proposed scheduling method in the project's final stage. The current implementation has been tested using datasets inspired by ITC'02 benchmark circuits, specifically d695 and p22810. We compared the performance of Greedy, ABC, MACO, Bat, and Firefly algorithms based on makespan, peak power use, and execution time. The results show that metaheuristic-based scheduling techniques can significantly shorten the total test application time while meeting all specified limits, achieving up to 15 to 36 % improvement over the basic Greedy method. Our ongoing work focuses on complete 3D SoC modeling and hardware validation to boost the reliability and applicability of the proposed framework.

Contents

DECLARATION	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 PROJECT OVERVIEW	1
1.2 MOTIVATION	3
1.3 OBJECTIVES	4
1.4 NOVELTY OF WORK	5
1.5 CONTRIBUTIONS OF THE PROPOSED WORK	5
1.6 ASSUMPTIONS AND CONSTRAINTS	5
1.7 ORGANIZATION OF THE REPORT	6
2 LITERATURE SURVEY	7
2.1 LITERATURE SURVEY	7
2.2 RESEARCH GAPS	13
2.3 PROBLEM DEFINITION AND SCOPE	14
3 FLOW CHART	15
3.1 SYSTEM ARCHITECTURE	15
3.2 ANALYSIS	15
3.3 Wrapper Design Methodology	16

3.3.1	TAM Allocation Strategy	17
3.3.2	Test Scheduling Formulation	18
3.3.3	TSV-Aware Scheduling	19
3.3.4	Thermal-Aware Scheduling	20
3.3.5	Optimization Algorithms	20
3.4	TOOLS AND TECHNOLOGIES USED	23
4	PROJECT DESIGN AND DESCRIPTION	24
4.1	DESCRIPTION	24
4.2	U.G SUBJECTS UTILIZED	24
4.3	STANDARDS USED	25
5	IMPLEMENTATION & EXPERIMENTAL RESULT	27
5.1	MATHEMATICAL FORMULATION	27
5.1.1	Objective Function	27
5.1.2	Power Constraint	27
5.1.3	TAM Width Constraint	27
5.2	BENCHMARK DATA ANALYSIS	28
5.3	SIMULATION RESULTS AND DISCUSSION	28
5.3.1	Makespan Evaluation	28
5.3.2	Peak Power Constraints Verification	28
5.3.3	Algorithm Execution Runtime	29
5.3.4	Overall Makespan Reduction Summary	30
5.4	VISUALIZATION OF RESULTS	30
5.5	OVERALL SYNTHESIS OF RESULTS	31
6	OUTCOME AND PROSPECTIVE LEARNING	32
6.1	SCOPE AND OUTCOMES	32
6.2	PROSPECTIVE LEARNINGS	32
6.3	CONCLUSION	32
7	PROJECT TIMELINE	33
7.1	WORK BREAKDOWN & GANTT CHART	33
7.2	PROJECT TIMELINE	34
	REFERENCES	35

List of Tables

5.1	Test Makespan Comparison (Total Test Application Time)	28
5.2	Peak Power Consumption Analysis	29
5.3	Algorithm Execution Runtime Comparison	29
5.4	Overall Makespan Reduction Summary	30

List of Figures

1.1	Overall Flow Of Proposed 3D SoC Test Scheduling Framework	2
3.1	3D SOC Integrated Testing & Validation Flow (Horizontal Flow Chart)	15
3.2	Wrapper architecture for embedded core testing.	16
3.3	Structural distribution of global TAM bandwidth across parallel cores.	17
3.4	TSV communication structure in a multi-layer 3D SoC.	19
3.5	Cross-sectional view of localized thermal hotspot generation in a 3D SoC stack.	20
3.6	MACO scheduling execution loop flowchart.	21
3.7	Firefly Algorithm exploration pipeline flowchart.	22
3.8	Bat Algorithm echolocation search loop flowchart.	23
5.1	Algorithm Performance Comparison Dashboard	30
5.2	Gantt Charts for Best Schedules Across All Algorithms and Benchmarks	31
5.3	Stacked Power Consumption Across Parallel Test Sessions	31
7.1	Project Work Breakdown and Gantt Chart	33

LIST OF ABBREVIATIONS

TIET	Thapar Institute of Engineering and Technology
ECE	Electronics and Communication Engineering
ENC	Electronics and Computer Engineering
SoC	System on Chip
TSV	Through-Silicon Via
TAT	Total Test Application Time
TAM	Test Access Mechanism
MACO	Modified Ant Colony Optimization
ABC	Artificial Bee Colony

Chapter 1

INTRODUCTION

1.1 PROJECT OVERVIEW

A System on Chip (SoC) is a semiconductor-based device that incorporates various types of integrated electronic components, including processing units, memory units, interface units, and other specialized hardware units in one piece of equipment. Despite the fact that recent SoC advances have enabled improved performance and functionality, they have also contributed to the complexity of testing. An increase in the number of embedded processor cores results in higher volumes of test data and resources being utilized. Testing is an essential stage in the SoC design cycle because it ensures the reliability and quality of the manufactured chip.

One of the primary goals of SoC testing is to minimize the Total Test Application Time (TAT) while efficiently utilizing available testing resources. However, reducing test time must be achieved without violating important constraints such as power consumption and Test Access Mechanism (TAM) width [2]. Excessive power consumption during testing may cause thermal issues, whereas inefficient TAM allocation can increase testing overhead and cost.

This project focuses on developing an optimized test scheduling framework that reduces test application time while satisfying power and TAM constraints. To achieve this objective, several optimization techniques, including Greedy scheduling, Artificial Bee Colony (ABC), Modified Ant Colony Optimization (MACO), Bat Algorithm, and Firefly Algorithm, are studied and compared. The proposed work also provides a foundation for future extension towards TSV-aware 3D SoC testing and hardware-assisted validation.

3D SoCs face massive challenges in minimizing Total Test Access Time (TAT), such as high power consumption during testing which leads to thermal hotspots that damage the chip. As 3D integration increases complexity through the use of Through-Silicon Vias (TSVs), traditional testing methods lead to excessive TAT and significant routing costs [5],[13]. 3D stacking increases test challenges: routing complexity, TSV faults, thermal hotspots, and high-power density inherent in stacked architectures imposes severe thermal constraints.

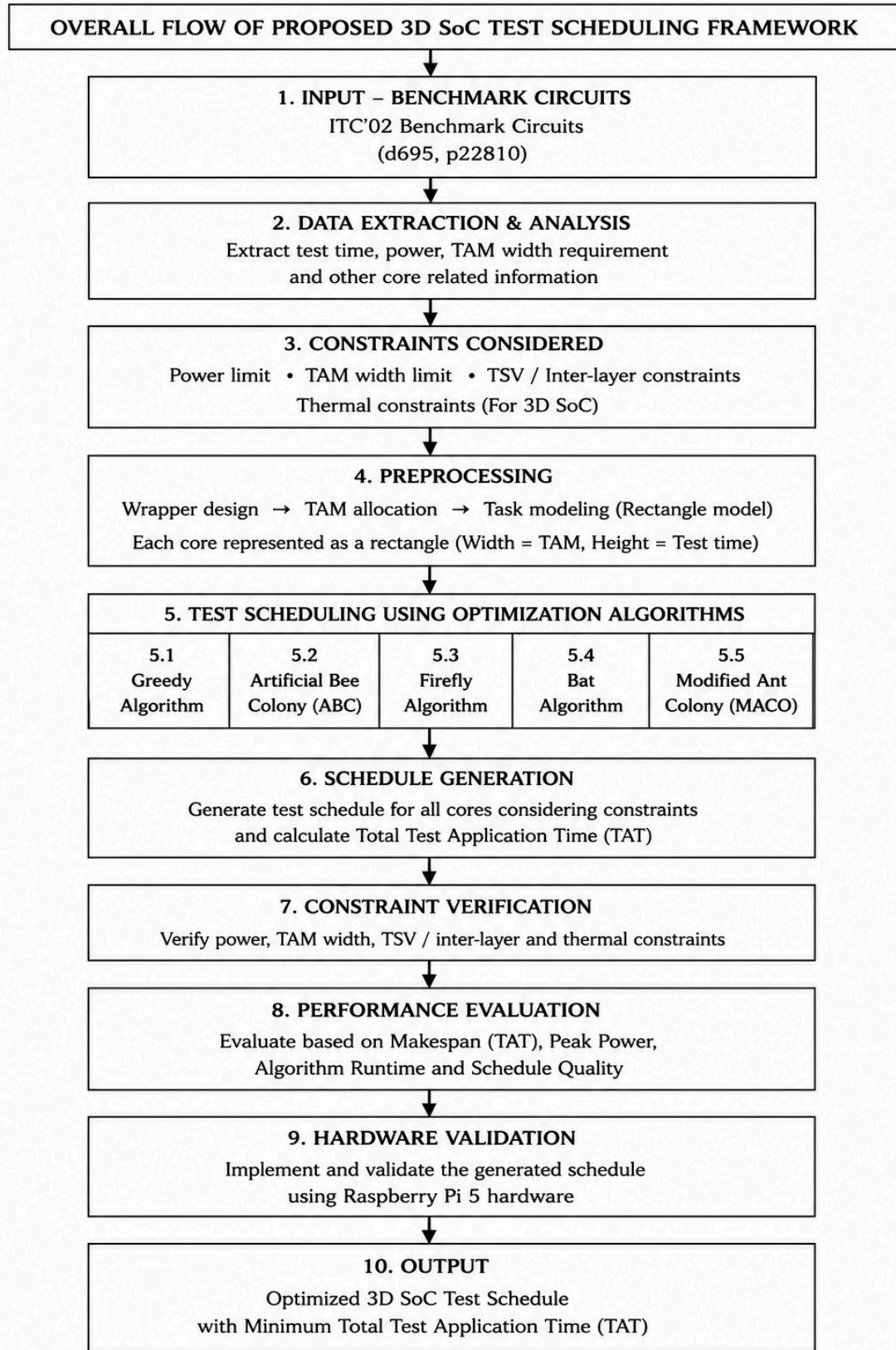


Figure 1.1: Overall Flow Of Proposed 3D SoC Test Scheduling Framework

Figure 1.1 presents the overall workflow of the proposed SoC test scheduling framework. The process begins with benchmark dataset analysis, followed by wrapper design and TAM allocation. Test schedules are then generated using various optimization algorithms, including Greedy, ABC, MACO, Bat, and Firefly algorithms. The generated schedules are evaluated using makespan, peak power consumption, and execution time as performance metrics. The framework is further intended to be extended towards 3D SoC testing and hardware-assisted validation in future work.

1.2 MOTIVATION

Larger test time is consumed due to large test data and increasing design intricacy for SoCs. The major problem in optimum test solution design is reducing the testing time while not exceeding safe thermal limitations.

In recent years, researchers have applied various optimization algorithms to improve test scheduling performance. Algorithms such as Artificial Bee Colony (ABC), Ant Colony Optimization (ACO), Firefly Algorithm, and Bat Algorithm have shown promising results in reducing testing time while satisfying power constraints [1],[4],[8]. These algorithms belong to a class of techniques known as metaheuristic optimization algorithms, which search for near-optimal solutions within a large solution space.

Although many of these approaches achieve significant improvements in test scheduling, most existing studies focus primarily on reducing test time. Only a limited number of works consider additional constraints such as thermal effects, TSV utilization, and real hardware validation. These limitations become more critical in the case of 3D SoCs, where heat dissipation and inter-layer communication play a major role in system reliability [6],[10].

Modern SoC testing involves several conflicting requirements. Although minimizing the Total Test Application Time (TAT) is a primary objective in SoC testing, it often leads to increased resource utilization and power consumption. Conversely, enforcing strict power constraints may result in longer testing durations. Therefore, an effective test scheduling framework must achieve an optimal balance between testing efficiency and system limitations.

1.3 OBJECTIVES

To solve the trade-off between test speed and thermal safety, we aim to design an optimized test scheduling algorithm. The project is carried out in an iterative phase-wise approach:

- **Phase 1: 2D SoC Test Optimization:** Develop and implement an optimized test scheduling solution for 2D System-on-Chip (SoC) architectures to minimize total test application time while strictly adhering to multidimensional limits, including time, peak power, and TAM (Test Access Mechanism) width constraints.
- **Phase 2: 3D SoC Test Scaling & Management:** Extend the optimized test methodologies to complex 3D stacked SoCs by integrating robust constraint-management frameworks. This phase ensures efficient scheduling under time, power, TAM, and TSV constraints while preventing localized thermal hot-spots during testing.
- **Phase 3: Real-World Hardware Validation:** Conduct empirical validation using an external hardware tester (Raspberry Pi 5) to rigorously verify the theoretical algorithmic performance metrics and constraint management against actual physical hardware limitations [15].

The major challenges addressed in this work include:

- Minimization of Total Test Application Time (TAT).
- Efficient allocation of Test Access Mechanism (TAM) resources.
- Compliance with peak power constraints during testing.
- Development of optimized scheduling strategies using metaheuristic algorithms.
- Extension of the scheduling framework towards advanced 3D SoC environments.

Based on these challenges, the primary objectives of this project are:

- To develop an efficient and constraint-aware SoC test scheduling framework.
- To evaluate and compare different optimization algorithms for test scheduling.
- To minimize makespan while satisfying power and TAM width constraints.
- To investigate the applicability of metaheuristic optimization techniques for future 3D SoC testing.
- To establish a foundation for future hardware-assisted validation using Raspberry Pi 5.

1.4 NOVELTY OF WORK

The novelty lies in bridging the gap between theoretical software-based swarm optimization algorithms (like MACO, Bat, and Firefly) and actual physical hardware constraints by validating the generated test schedules directly on a Raspberry Pi 5 setup.

1.5 CONTRIBUTIONS OF THE PROPOSED WORK

The major contributions of this project are summarized as follows:

- Development of a power-aware and constraint-aware test scheduling framework for SoC testing.
- Comparative evaluation of Greedy, ABC, MACO, Bat, and Firefly algorithms using benchmark datasets.
- Analysis of test scheduling performance based on makespan, peak power consumption, and execution time.
- Integration of wrapper design and TAM allocation concepts within the scheduling framework.
- Establishment of a foundation for future TSV-aware 3D SoC scheduling and hardware validation.

1.6 ASSUMPTIONS AND CONSTRAINTS

The proposed scheduling framework is developed under a set of predefined constraints to ensure realistic and feasible test schedules. The core constraints defined for this optimization problem include:

- **Power Limits / Constraint:** The cumulative power consumption of concurrently scheduled tests must remain within the specified power budget.
- **TAM Width Constraints:** Ensuring the total allocated TAM width should not exceed the available TAM resources.
- **Scheduling Constraint:** Test schedules should minimize the overall makespan while satisfying all imposed limitations.

- **TSV Constraints / 3D SoC Constraint (Future Extension):** Managing inter-die communication overhead and incorporating TSV-aware scheduling considerations during the later stages of the project.

1.7 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows.

- **Chapter 2** presents the literature review of existing SoC test scheduling techniques and identifies the associated research gaps.
- **Chapter 3** describes the proposed methodology, system architecture, and overall workflow of the scheduling framework.
- **Chapter 4** discusses the implementation details, including wrapper design, TAM allocation, and optimization algorithms.
- **Chapter 5** presents the experimental results along with a detailed performance analysis and discussion.
- **Chapter 6** summarizes the conclusions of the work and outlines future directions for 3D SoC extension and hardware-assisted validation.

Chapter 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

Paper 1 Title: Test Scheduling and Test Time Reduction for SoC by Using Enhanced Firefly Algorithm

Authors: Gokul Chandrasekaran, Gopinath Singaram, Rajkumar Duraisamy, Akash Sanjay Ghodake, and Parthiban Kunnathur Ganesan [1]

The authors proposed an Enhanced Firefly Algorithm (EFA) to optimize test scheduling in System-on-Chip (SoC) testing. The primary objective of the work was to minimize the Total Test Application Time (TAT) while satisfying power and resource constraints. The proposed algorithm is based on the flashing and attraction behavior of fireflies, where less attractive solutions move toward better solutions to achieve an optimal schedule. The enhanced version improves the search capability and convergence speed of the conventional Firefly Algorithm. Experimental results showed that the proposed method significantly reduced test application time and generated better scheduling solutions compared to traditional approaches. The study concluded that the Enhanced Firefly Algorithm is an efficient and reliable optimization technique for SoC test scheduling and can effectively handle complex testing constraints.

Paper 2 Title: Survey on Power-Aware Test Scheduling and Scan Chain Optimization in SoC Testing

Authors: International Journal of Advanced Research in Computer Science (2025) [2]

The authors presented a comprehensive survey on power-aware test scheduling and scan chain optimization techniques used in SoC testing. The paper discusses the challenges of excessive power consumption during testing, which can lead to thermal hotspots and reliability issues.

Various scheduling methods and scan-chain optimization techniques were reviewed to reduce testing power while maintaining acceptable test application times. The survey highlights the importance of balancing power consumption, test time, and hardware overhead in modern SoC designs. The authors concluded that intelligent scheduling and optimization algorithms are essential for achieving efficient and reliable SoC testing.

Paper 3 Title: Using SystemC TLM Modeling To Solve AI Data Movement Challenges

Authors: SemiEngineering (2026) [3]

The authors proposed the use of SystemC Transaction-Level Modeling (TLM) to solve data movement challenges in AI-based hardware systems. The work focuses on improving communication efficiency between processors, memory units, and other system components. By using high-level modeling, designers can analyze bandwidth requirements, communication delays, and resource utilization before hardware implementation. The study demonstrated that TLM-based modeling reduces development complexity, enables faster simulation, and improves overall system performance. The authors concluded that SystemC TLM is a powerful tool for system-level design exploration and validation.

Paper 4 Title: Test Scheduling and Test Time Minimization of System-on-Chip Using Modified BAT Algorithm

Authors: IEEE Xplore (2022) [4]

The authors proposed a Modified BAT Algorithm (MBA) for optimizing test scheduling in System-on-Chip architectures. The algorithm is inspired by the echolocation behavior of bats and uses adaptive frequency, velocity, and loudness parameters to search for optimal solutions. The main objective was to minimize Total Test Application Time while satisfying power and resource constraints. Experimental results showed that the Modified BAT Algorithm achieved shorter test schedules and better resource utilization than conventional scheduling methods. The study concluded that BAT-based optimization techniques can effectively solve complex

SoC test scheduling problems and improve overall testing efficiency.

Paper 5 Title: Test Architecture Design and Optimization for Three-Dimensional SoCs

Authors: Li Jiang, Lin Huang, and Qiang Xu [5]

The authors proposed a test architecture design and optimization framework for three-dimensional System-on-Chip (3D SoC) architectures. The study focused on addressing the challenges introduced by vertical integration, such as Through-Silicon Via (TSV) utilization, routing complexity, and increased testing overhead. An optimized Test Access Mechanism (TAM) architecture was developed to improve test data transportation between stacked dies. The proposed method aimed to reduce test application time while efficiently utilizing available testing resources. Experimental results demonstrated that optimized test architecture design can significantly improve testing efficiency and reduce testing cost in 3D SoC environments. The work established an important foundation for future research in 3D SoC testing and scheduling.

Paper 6 Title: Thermal-Aware Task Scheduling for 3D-Network-on-Chip: A Bottom-to-Top Scheme

Authors: Y. Cui, W. Zhang, V. Chaturvedi, W. Liu, and B. He [6]

The authors proposed a thermal-aware task scheduling approach for 3D Network-on-Chip systems. The study recognized that the stacking of multiple dies increases power density, leading to thermal hotspots that can degrade system performance and reliability. To address this issue, a Bottom-to-Top scheduling scheme was introduced to distribute workloads efficiently across different layers of the 3D architecture. The proposed method reduced peak temperature while maintaining system performance. Experimental evaluations showed significant improvements in thermal distribution and energy efficiency compared to conventional scheduling techniques. The study highlighted the importance of thermal management in modern 3D integrated circuits.

Paper 7 Title: Minimization of Test Time in System on Chip Using Artificial Intelligence-Based Test Scheduling Techniques

Authors: Gokul Chandrasekaran, Sakthivel Periyasamy, and Karthikeyan Panjappagounder Rajamanickam [7]

The authors presented an Artificial Intelligence-based approach for minimizing test time in System-on-Chip architectures. The proposed technique utilized intelligent optimization strategies to determine efficient scheduling sequences for core testing. The methodology considered various testing constraints, including power consumption and resource allocation, while attempting to minimize Total Test Application Time (TAT). The results demonstrated that AI-based scheduling methods outperform conventional heuristic approaches by generating more efficient test schedules. The study concluded that intelligent optimization techniques have significant potential for solving complex SoC testing problems and improve overall testing efficiency.

Paper 8 Title: An Empirical Investigation of Metaheuristic and Heuristic Algorithms for a 2D Packing Problem

Authors: E. Hopper and B.C.H. Turton [8]

In this paper, the authors investigated various heuristic and metaheuristic algorithms for solving two-dimensional packing problems. The objective was to arrange rectangular objects within a fixed area while minimizing unused space. Several optimization techniques were analyzed and compared based on solution quality and computational efficiency. The study demonstrated that metaheuristic approaches are capable of producing near-optimal solutions for complex optimization problems. The concepts presented in this work later became useful for SoC test scheduling, where test rectangles are packed under power and TAM constraints. The research provided a theoretical basis for applying swarm intelligence and optimization algorithms in scheduling applications.

Paper 9 Title: Test Scheduling and Test Time Reduction for SoC by Using Enhanced Firefly Algorithm

Authors: Gokul Chandrasekaran, Gopinath Singaram, Rajkumar Duraisamy, Akash Sanjay Ghodake, and Parthiban Kunnathur Ganesan [9]

The authors proposed an Enhanced Firefly Algorithm (EFA) for optimizing test scheduling in System-on-Chip testing. The algorithm was inspired by the flashing behavior of fireflies and utilized attraction-based search mechanisms to identify optimal scheduling solutions. The primary objective was to minimize Total Test Application Time while satisfying power and testing resource constraints. The enhanced version improved the exploration and exploitation capabilities of the conventional Firefly Algorithm, leading to faster convergence and better solution quality. Experimental results showed a considerable reduction in test application time compared to traditional scheduling methods. The study concluded that the Enhanced Firefly Algorithm is an effective optimization technique for handling complex SoC testing challenges and improving overall scheduling efficiency.

Paper 10 Title: Thermal-Aware Test Scheduling and Floor Planning for Three-Dimensional Stacked Integrated Circuits

Authors: N. S. M. Shah et al. [10]

The authors proposed a thermal-aware test scheduling and floor-planning methodology for 3D stacked integrated circuits. The work focuses on reducing thermal hotspots generated during the testing process. A combined scheduling and floor-planning strategy was used to distribute test activities efficiently across different layers of the chip. Experimental results showed improved thermal distribution and enhanced reliability of 3D integrated circuits. The study concluded that thermal-aware scheduling plays a crucial role in maintaining chip performance and reducing overheating issues during testing.

Paper 11 Title: Test-Path Scheduling for Interposer-Based 2.5D Integrated Circuits Using an Orthogonal Learning-Based Differential Evolution Algorithm

Authors: Y. Zhang et al. [11]

This paper introduced an Orthogonal Learning-Based Differential Evolution Algorithm for test-path scheduling in 2.5D integrated circuits. The proposed method optimizes test paths while considering communication and routing constraints. The algorithm improved convergence speed and generated high-quality scheduling solutions. Experimental results showed better performance compared to traditional differential evolution approaches.

Paper 12 Title: Machine Learning-Based Predictive Thermal Estimation and Frequency-Scaled Test Scheduling for 3D ICs

Authors: S. Roy and P. Giri [12]

The authors proposed a machine learning-based framework for predictive thermal estimation and frequency-scaled test scheduling in 3D integrated circuits. Machine learning models were used to predict temperature variations during testing. Based on the predicted thermal conditions, test frequencies were dynamically adjusted to prevent overheating. The proposed approach successfully reduced thermal hotspots and improved testing reliability.

Paper 13 Title: A Metaheuristic Optimization Approach for Through-Silicon Via (TSV) Test Scheduling in 3D Integrated Circuits

Authors: A. Verma and R. Kumar [13]

The authors proposed a metaheuristic optimization technique specifically for TSV test scheduling in 3D integrated circuits. The approach focused on reducing TSV testing overhead while maintaining testing reliability. The study highlighted the importance of TSV-aware scheduling in modern 3D architectures and achieved improved test efficiency.

Paper 14 Title: Constraint-Driven Test Scheduling in 3D Stacked ICs via Swarm Optimization Techniques

Authors: L. Zheng and Q. Xu [14]

The authors presented a swarm optimization-based approach for constraint-driven test scheduling in 3D stacked integrated circuits. The proposed methodology considers multiple constraints such as power consumption, TAM width, and thermal limitations. Results demonstrated improved scheduling quality and reduced overall testing time.

Paper 15 Title: Hardware-in-the-Loop Validation for Dynamic Thermal-Aware VLSI Testing Architectures

Authors: S. Patel and M. Desai [15]

The authors proposed a Hardware-in-the-Loop (HIL) validation framework for thermal-aware VLSI testing architectures. The methodology integrates software simulations with hardware platforms to validate testing strategies under real-world conditions. Experimental evaluations confirmed the effectiveness of thermal-aware scheduling techniques and improved testing reliability.

2.2 RESEARCH GAPS

- Traditional static scheduling methods are insufficient for the dynamic thermal complexities of modern 3D integration.
- Existing algorithms often optimize for test time without accurately accounting for real-world interface lags and TSV thermal choke-points.

2.3 PROBLEM DEFINITION AND SCOPE

The scope of this project is to model 2D and 3D SoCs, apply advanced metaheuristics to schedule tests under rigid constraints, and finally map these results to hardware (Raspberry Pi 5) for validation.

Chapter 3

FLOW CHART

3.1 SYSTEM ARCHITECTURE

The architecture involves a phase-wise methodology. **Phase 1: Data Preparation** comprises SoC analysis and Constraint analysis (Step 1 to Step 4). **Phase 2: Scheduling Progression** comprises Core level test scheduling, progressing to 2D test scheduling, studying TSV constraints, generating 3D test scheduling, and ultimately Machine Learning integration.

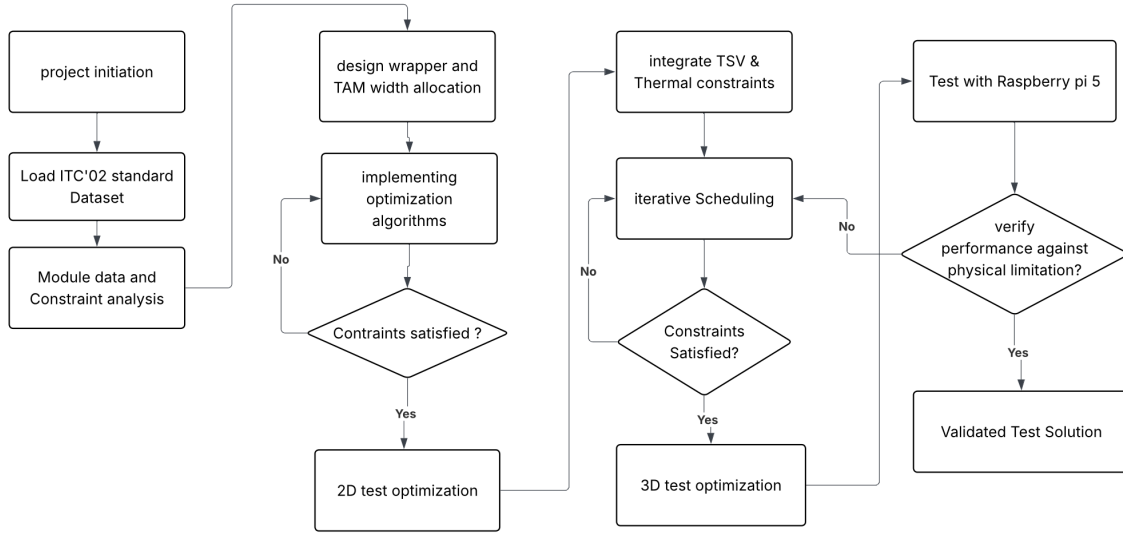


Figure 3.1: 3D SOC Integrated Testing & Validation Flow (Horizontal Flow Chart)

3.2 ANALYSIS

Testing relies on solving a 2D Bin Packing problem using wrappers. The goal is to pack all rectangles (cores) into a bin of fixed width (SOC pin count) while minimizing the bin's height (TAT).

3.3 Wrapper Design Methodology

To effectively decouple embedded cores during the structural testing cycle, this framework adheres to the **IEEE 1500 Standard for Embedded Core Test**. The modularity of modern SoCs is exploited by isolating each intellectual property (IP) core using test wrapper architectures.

- **Wrapper Cells:** Positioned at the functional boundary of each core to isolate internal logic from surrounding digital logic during test execution.
- **Scan Chains:** Internal scan paths are balanced and configured into optimal lengths to shift in test stimuli and shift out test responses efficiently.
- **Embedded Core Testing:** Enables modular expansion, facilitating independent core diagnostics without requiring physical access to every internal node.

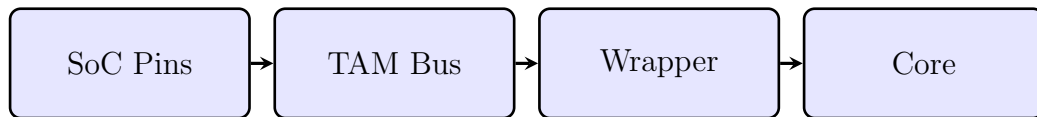


Figure 3.2: Wrapper architecture for embedded core testing.

The functional interaction detailed in Figure 3.2 illustrates how external test access resources are routed to individual cores. The wrapper acts as an interface that maps the external Test Access Mechanism (TAM) width onto internal scan chains. By optimizing the wrapper design configuration, the total scan chain lengths are minimized, reducing the serialization latency of test vector delivery and establishing a modular footprint for multi-core configurations.

3.3.1 TAM Allocation Strategy

The Test Access Mechanism (TAM) acts as the physical highway system responsible for routing test stimuli from external tester pins to the core wrappers and returning the test responses to output pins. Efficient resource sharing requires smart partitioning of the globally available TAM bus across multiple concurrent cores.

- **TAM Width:** Represents the overall bit-width capacity of the test bus layout.
- **Core Allocation:** Dynamically assigns specific bit-slices of the primary TAM bus to corresponding internal wrappers to optimize bandwidth utility.
- **Resource Sharing:** Parallelizes core testing execution profiles while ensuring no duplicate line allocation contentions occur during a single test interval.

The total cumulative allocation of individual TAM channel widths assigned across all active cores at any parallel time instance (t) must strictly fit within the maximum globally budgeted hardware TAM pin count (TAM_{max}):

$$\sum_{i \in \text{Active}} TAM_i \leq TAM_{max} \quad (3.1)$$

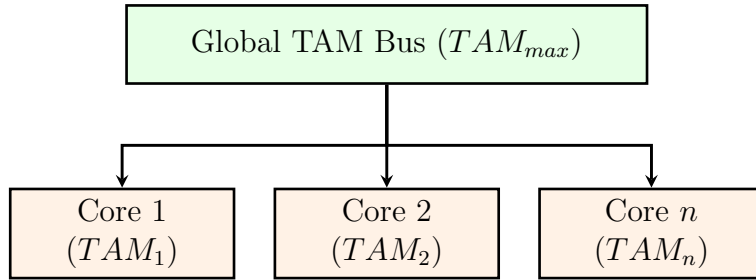


Figure 3.3: Structural distribution of global TAM bandwidth across parallel cores.

As diagrammed in Figure 3.3, the system splits the primary global TAM resource pool into specific slice distributions. Because different cores have varying scan chain lengths and test data volumes, the allocation framework evaluates the optimal rectangular fit of these TAM widths alongside test cycles. This balances the overall structure, transforming test scheduling into a multi-constraint 2D bin-packing problem [8].

3.3.2 Test Scheduling Formulation

The test scheduling problem is formally modeled as a highly constrained multi-dimensional optimization problem. The primary objective is to minimize the **Total Test Application Time (TAT)**, often called the **makespan** of the test profile.

$$\text{Objective: } \min(\text{TAT}) \quad (3.2)$$

To prevent physical damage, routing congestion, or structural data collisions, every generated schedule must strictly satisfy the following operational constraints at any given execution time step t :

1. **Power Constraint:** Ensures concurrent test sessions do not exceed structural limits.

$$\sum_{i \in \text{Active}(t)} P_i \leq P_{max} \quad (3.3)$$

Where P_i is the peak power consumption of active core i , and P_{max} is the safe continuous power threshold.

2. **TAM Width Constraint:** Restricts global test bus configurations.

$$\sum_{i \in \text{Active}(t)} TAM_i \leq TAM_{max} \quad (3.4)$$

3. **TSV Constraint (3D Specific):** Limits vertical inter-die structural routing channels.

$$\sum_{j \in \text{Active_TSV}(t)} TSV_j \leq TSV_{max} \quad (3.5)$$

Where TSV_j represents active vertical interconnect tracks, and TSV_{max} represents the maximum allowable TSV bandwidth limit.

By analyzing these mathematical equations, the optimization engine avoids scheduling conflicting, high-power test behaviors simultaneously. This mitigates systemic power spikes and prevents routing over-allocation during parallel execution sessions.

3.3.3 TSV-Aware Scheduling

Transitioning from 2D planar chips to 3D stacked integrated circuits relies on vertical interconnects known as **Through-Silicon Vias (TSVs)** [13]. While TSVs offer dense inter-die connectivity, they introduce unique manufacturing defect profiles, routing bottlenecks, and thermal choke-points.

- **TSV Architecture:** Structural vertical micro-vias etching through silicon substrates to connect active device layers.
- **TSV Fault Profiles:**
 - *Open Faults:* Micro-cracks or voids causing complete electrical discontinuities.
 - *Short Faults:* Isolation layer breakdowns leading to bridging shorts between adjacent TSVs.
 - *Delay Faults:* Resistive anomalies altering signal propagation times across die boundaries.

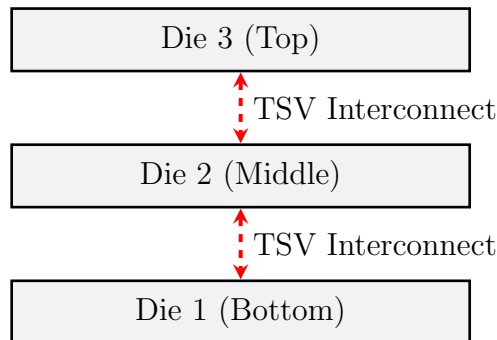


Figure 3.4: TSV communication structure in a multi-layer 3D SoC.

The multi-tiered vertical configuration in Figure 3.4 illustrates the structural complexity of 3D integration. Because TSVs are prone to localized mechanical stress and processing defects, the 3D-aware scheduling engine monitors the volume of data passing vertically between active tiers. It tracks and bounds the total concurrent signals to minimize signal cross-talk and avoid routing bottlenecks.

3.3.4 Thermal-Aware Scheduling

A major bottleneck in 3D integration is high power density. Stacking multiple active dies drastically reduces the available surface-to-volume ratio for heat dissipation, trapping thermal energy within the inner layers. Without active mitigation, parallel testing triggers intense localized thermal hotspots.

- **Thermal Hotspots:** Highly localized regions experiencing sharp temperature spikes due to clustered power-hungry test operations.
- **Power Density:** The concentration of heat dissipation per unit area increases exponentially with each added active vertical die layer.
- **Thermal Constraints:** Safe operational bounds enforced to keep the transient junction temperature (T_j) below a critical threshold (T_{limit}) to protect the chip from structural cracking or accelerated degradation.

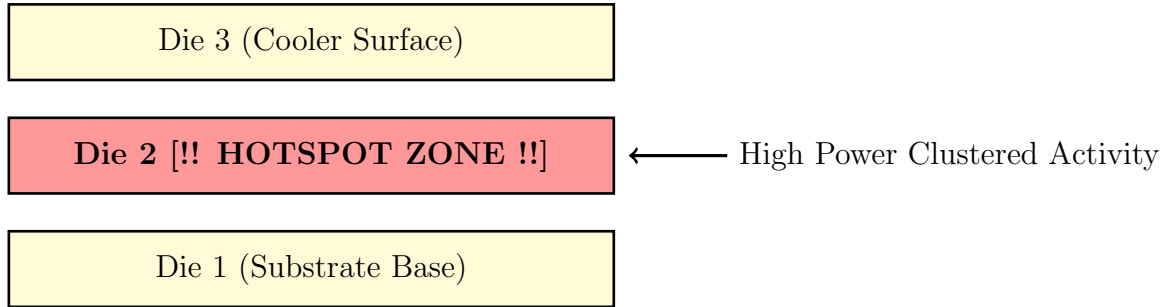


Figure 3.5: Cross-sectional view of localized thermal hotspot generation in a 3D SoC stack.

As illustrated in Figure 3.5, heat generated in the middle layers (e.g., Die 2) becomes trapped, creating a dangerous thermal profile. The scheduling engine handles this by spatial-temporal decoupling: it systematically spreads high-power test vectors across different time frames and distinct layers to ensure a uniform, safe thermal gradient during test execution [6],[10].

3.3.5 Optimization Algorithms

Modified Ant Colony Optimization (MACO)

The Modified Ant Colony Optimization algorithm mimics the foraging behavior of natural ant colonies, enhanced with a dynamic population scaling engine. Traditional ACO struggles with

stagnation or slow convergence when the remaining core count drops. MACO overcomes this by scaling the ant population dynamically based on resource availability.

Operational Steps and Working:

1. Initialize global pheromone trails (τ_{ij}) across all core sequence pathways.
2. Deploy an initial ant colony to construct valid 2D/3D test schedules matching power and TAM limits.
3. Calculate path selection probability using pheromone concentration (τ_{ij}) and heuristic visibility (η_{ij}) :

$$P_{ij} = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum (\tau_{ik})^\alpha \cdot (\eta_{ik})^\beta} \quad (3.6)$$

4. **Dynamic Scaling Step:** Monitor the core pool size; automatically adjust active worker counts to maintain search momentum and prevent local minima trapdoors.
5. Evaporate global pheromones and reinforce elite paths with localized deposition updates.

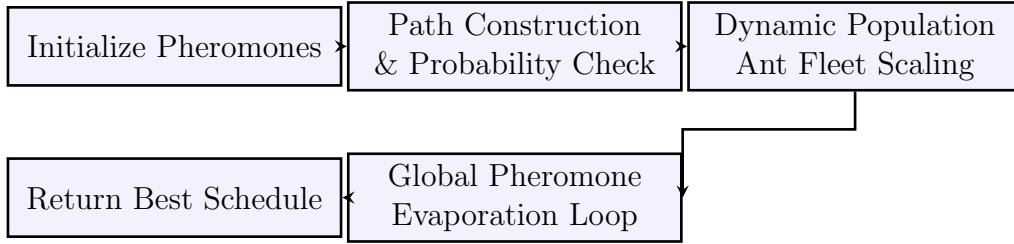


Figure 3.6: MACO scheduling execution loop flowchart.

Firefly Algorithm (FA)

The Firefly Algorithm is a nature-inspired metaheuristic based on the flashing light behavior [1] and mutual attraction patterns of fireflies. In this context, a firefly's coordinates represent a specific test sequence configuration, while its perceived light brightness maps inversely to the total test makespan.

Operational Steps and Working:

1. Initialize a population of fireflies randomly across the scheduling solution space.
2. Formulate light intensity I for each firefly relative to its fitness score (lower makespan = higher brightness).

3. Evaluate the spatial distance (r_{ij}) between firefly i and firefly j .
4. Compute the changing light attractiveness (β) using the distance variable:

$$\beta(r) = \beta_0 \cdot e^{-\gamma r^2} \quad (3.7)$$

Where β_0 represents original source attractiveness, and γ is the fixed light absorption coefficient.

5. Move the less bright firefly i toward the brighter firefly j using the attraction vector:

$$x_i^{(t+1)} = x_i^{(t)} + \beta_0 e^{-\gamma r_{ij}^2} (x_j^{(t)} - x_i^{(t)}) + \alpha_t \cdot \epsilon_i \quad (3.8)$$

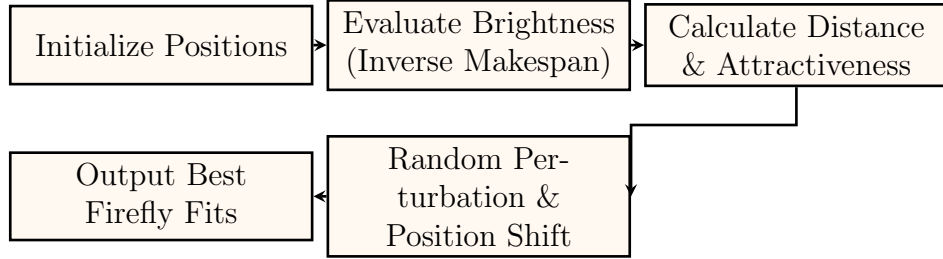


Figure 3.7: Firefly Algorithm exploration pipeline flowchart.

Bat Algorithm (BA)

The Bat Algorithm leverages the echolocation capabilities of microbats to navigate and hunt [4]. Individual bat positions map to test schedules, utilizing variable sound emission frequencies, adjustable pulsing velocities, and changing loudness levels to toggle between global exploration and local exploitation.

Operational Steps and Working:

1. Initialize bat positions x_i , search velocities v_i , and sound pulse emission frequencies f_i .
2. Dynamically scale loop stepping frequencies using boundary limits:

$$f_i = f_{min} + (f_{max} - f_{min}) \cdot \beta \quad (3.9)$$

3. Update velocity vectors and position maps based on the current global best layout (x_*):

$$v_i^{(t)} = v_i^{(t-1)} + (x_i^{(t-1)} - x_*) \cdot f_i \quad (3.10)$$

$$x_i^{(t)} = x_i^{(t-1)} + v_i^{(t)} \quad (3.11)$$

4. Apply localized random walks near the global best candidate if the random draw exceeds the active pulse rate (r_i).
5. Adjust loudness (A_i) and pulse emission rates (r_i) as convergence approaches:

$$A_i^{(t+1)} = \alpha A_i^{(t)}, \quad r_i^{(t+1)} = r_i^{(0)} \cdot [1 - e^{-\gamma t}] \quad (3.12)$$

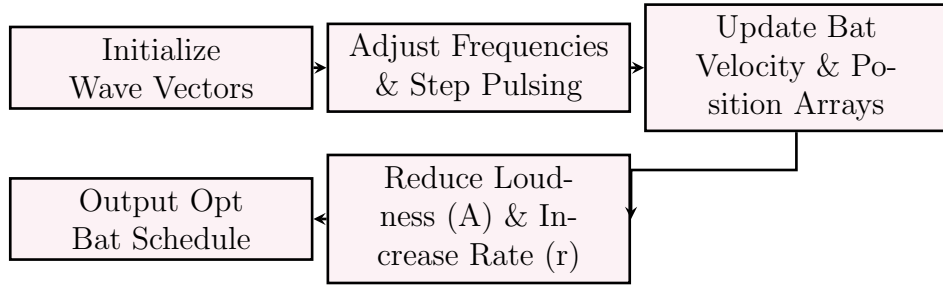


Figure 3.8: Bat Algorithm echolocation search loop flowchart.

3.4 TOOLS AND TECHNOLOGIES USED

- Python (Pandas, Numpy, Matplotlib)
- Optimization Frameworks (Bat, MACO, Firefly, ABC algorithms)
- ITC'02 Benchmark Datasets (d695, p22810)
- Hardware Integration: Raspberry Pi 5

Chapter 4

PROJECT DESIGN AND DESCRIPTION

4.1 DESCRIPTION

The scheduling algorithms take core power metrics and test cycles as inputs. They utilize swarm intelligence frameworks to iterate through possible TAM distribution sequences, returning the schedule that possesses the lowest makespan without crossing the designated continuous power threshold.

4.2 U.G SUBJECTS UTILIZED

- **VLSI Design and Testing:** Provided the fundamental understanding of scan chain architectures and the IEEE 1500 standard necessary for core isolation.
- **Computer Architecture and Organization:** Supported the analysis of SoC-level components, multi-core resource distribution, and the logic behind TAM bus allocation.
- **Thermal Management in Electronics:** Critical for understanding heat dissipation constraints and the mechanics of preventing localized thermal hotspots in 3D-stacked architectures [10],[12].
- **Embedded Systems:** Enabled the design of the hardware-assisted validation framework and the successful implementation of test schedules on the Raspberry Pi 5 platform.
- **Optimization Techniques (Operations Research):** Provided the mathematical foundation for modeling the test scheduling problem as a multi-constrained 2D bin-packing challenge and applying swarm-based metaheuristics.
- **Data Structures and Algorithms:** Facilitated the efficient coding and iteration of scheduling algorithms like MACO, Bat, and Firefly to optimize makespan.

- **Microprocessors and Microcontrollers:** Applied to the real-world interfacing between the scheduling software and the physical hardware tester (Raspberry Pi 5).
- **Machine Learning (Prospective):** Relevant to future extensions of this work regarding predictive thermal estimation and advanced scheduling automation [12].

4.3 STANDARDS USED

IEEE 1500 Standard for Embedded Core Test

The IEEE 1500 standard provides a modular testing framework for embedded IP cores within an SoC environment through the following key functions:

- **Core Isolation:** Establishes a standardized hardware wrapper around individual cores to isolate them during testing, preventing structural interference between neighboring logic blocks.
- **Test Modes:** Utilizes dedicated wrapper internal circuitry to seamlessly switch components between normal operation and test modes (such as **InTest** and **ExTest**).
- **TAM Balancing:** Standardizes wrapper scan-chains to optimize internal register lengths, allowing scheduling algorithms to precisely calculate core execution time relative to assigned TAM widths [2],[7].

ITC'02 SoC Test Benchmarks

The International Test Conference 2002 (ITC'02) SoC Test Benchmarks serve as standardized, non-proprietary industry datasets utilized for evaluating testing algorithms through the following utilities:

- **Structured Core Profiles:** Provide complete structural data—including scan-chain counts, block configurations, and test details—for complex SoC designs such as the **d695** and **p22810** models.
- **Power & Time Constraints:** Define precise parameters for test sequences and maximum power dissipation, enabling exact constraint evaluation for multi-variable scheduling problems.

- **Performance Evaluation:** Act as a global baseline, allowing the optimization results generated by models like MACO, Bat, and Firefly to be universally verified and directly compared against existing literature.

Chapter 5

IMPLEMENTATION & EXPERIMENTAL RESULT

5.1 MATHEMATICAL FORMULATION

The 3D SoC test scheduling problem is modeled as a multi-constrained optimization problem to minimize Total Test Application Time (TAT) [14] under thermal power and Test Access Mechanism (TAM) bandwidth constraints.

Let the SoC consist of a set of N cores, $C = \{c_1, c_2, \dots, c_N\}$. For each core c_i : S_i is the scheduled start time, T_i is the test duration, P_i is the power consumption, and W_i is the allocated TAM width.

5.1.1 Objective Function

Minimize the total makespan (TAT):

$$TAT = \min \left(\max_{i \in \{1, \dots, N\}} (S_i + T_i) \right) \quad (5.1)$$

5.1.2 Power Constraint

Concurrently active cores at any time t must not exceed the peak system power limit (P_{max}):

$$\sum_{i \in \text{Active}(t)} P_i \leq P_{max} \quad \forall t \quad (5.2)$$

5.1.3 TAM Width Constraint

Total TAM lines utilized at time t must not exceed the maximum SoC TAM width (W_{max}):

$$\sum_{i \in \text{Active}(t)} W_i \leq W_{max} \quad \forall t \quad (5.3)$$

The set of active cores is defined as $\text{Active}(t) = \{c_i \mid S_i \leq t < S_i + T_i\}$.

5.2 BENCHMARK DATA ANALYSIS

Simulations used synthetic datasets modeled after the ITC'02 benchmarks:

- **d695 Benchmark:** 10 modules, total power = 76.48 W, total test cycles = 490,788. Power limit (P_{max}) = 45.89 W.
- **p22810 Benchmark:** 30 modules, total power = 156.75 W, total test cycles = 754,236. Power limit (P_{max}) = 94.05 W.

5.3 SIMULATION RESULTS AND DISCUSSION

5.3.1 Makespan Evaluation

Table 5.1 highlights the primary objective: minimizing the total test cycles across the evaluated scheduling algorithms.

Algorithm	d695 Benchmark (Cycles)	p22810 Benchmark (Cycles)	Performance vs. Greedy
Greedy (Baseline)	90,863	134,621	Baseline
Artificial Bee Colony (ABC)	62,527	124,427	Moderate
Bat Algorithm	58,126	114,344	Optimal
Modified ACO (MACO)	58,126	114,344	Optimal
Firefly Algorithm	58,126	114,344	Optimal

Table 5.1: Test Makespan Comparison (Total Test Application Time)

Discussion: The static Greedy approach results in heavily fragmented schedules and the highest makespan. Conversely, the Bat, MACO, and Firefly algorithms bypass local optima to achieve identical, optimal makespans [1],[4],[7]. This represents a $\sim 36\%$ reduction in TAT for the d695 benchmark and a $\sim 15\%$ reduction for the larger p22810 benchmark.

5.3.2 Peak Power Constraints Verification

Table 5.2 verifies that all generated schedules successfully maintain thermal safety by remaining below the specified power limits.

Algorithm	d695 Peak Power (W)	p22810 Peak Power (W)	Constraint Status (P_{max})
Greedy (Baseline)	43.29	89.43	Safe
Bat Algorithm	44.50	88.92	Safe
Artificial Bee Colony (ABC)	45.68	90.64	Safe
Modified ACO (MACO)	44.01	91.21	Safe
Firefly Algorithm	42.83	91.21	Safe

Table 5.2: Peak Power Consumption Analysis

Discussion: All algorithms successfully generated test schedules without violating the hard P_{max} constraints (45.89 W for d695; 94.05 W for p22810). While Firefly achieved the lowest peak power for d695, all swarm-intelligence techniques mathematically satisfy the thermal constraints formulated in Section 5.1 [14].

5.3.3 Algorithm Execution Runtime

Table 5.3 compares the computational overhead (software execution time) required by each algorithm to generate an optimal schedule.

Algorithm	d695 Time (s)	p22810 Time (s)	Swarm Speed Ranking
Greedy (Baseline)	0.0189	0.0031	N/A (Static)
Modified ACO (MACO)	3.5850	4.1554	1st (Fastest)
Firefly Algorithm	10.1183	5.1771	2nd
Artificial Bee Colony (ABC)	7.4205	8.6863	3rd
Bat Algorithm	10.0983	12.6920	4th

Table 5.3: Algorithm Execution Runtime Comparison

Discussion: The Greedy approach executes near-instantly due to its lack of iterative optimization, though its output is highly suboptimal. Among the meta-heuristics that hit the optimal makespan, MACO demonstrated superior efficiency. By dynamically scaling its population relative to unassigned cores, MACO converged significantly faster than the Firefly and Bat algorithms.

5.3.4 Overall Makespan Reduction Summary

Table 5.4 provides a high-level summary of the percentage improvements achieved by the optimal swarm algorithms compared to the traditional baseline approach.

Benchmark	Baseline (Greedy)	Best Makespan	Winning Algorithms	Overall Reduction
d695	90,863 Cyc.	58,126 Cyc.	Bat, MACO, Firefly	~ 36.03%
p22810	134,621 Cyc.	114,344 Cyc.	Bat, MACO, Firefly	~ 15.06%

Table 5.4: Overall Makespan Reduction Summary

5.4 VISUALIZATION OF RESULTS

Figure 5.1 — Algorithm Performance Comparison Dashboard

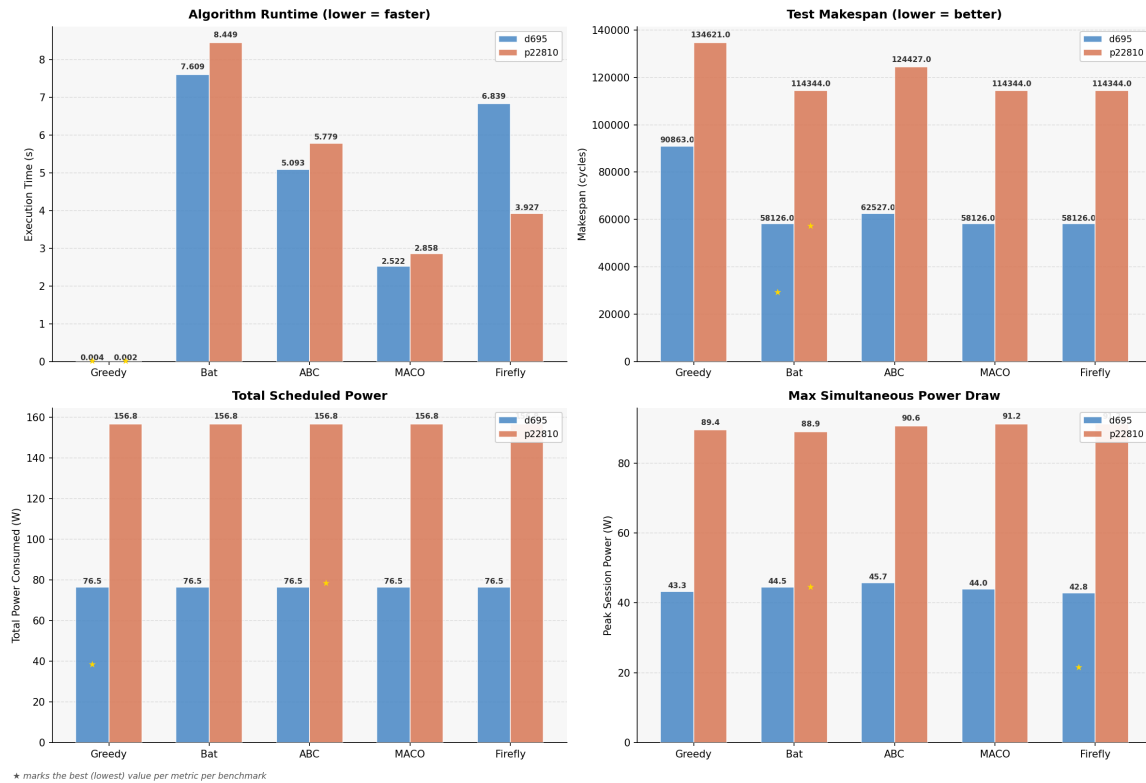


Figure 5.1: Algorithm Performance Comparison Dashboard

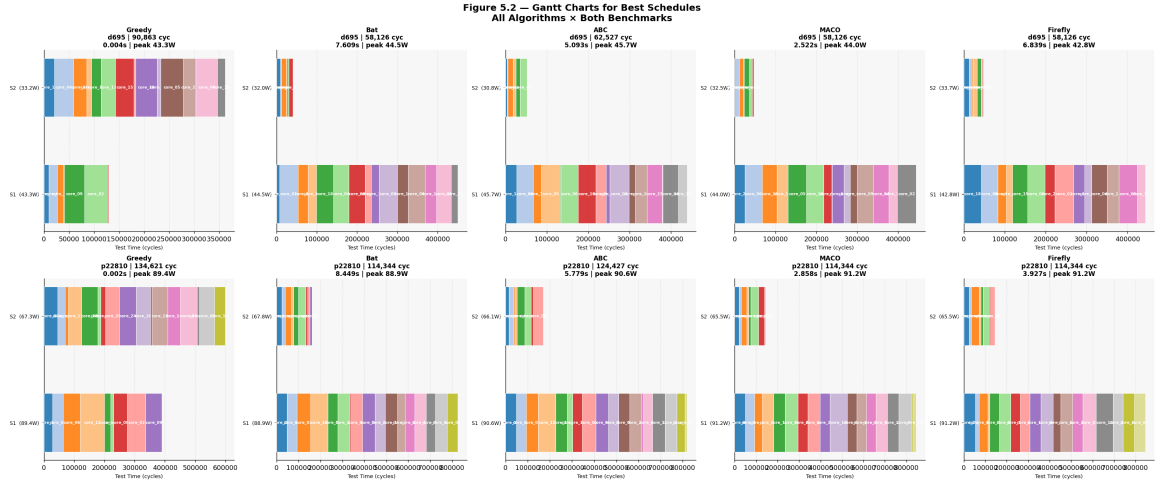


Figure 5.2: Gantt Charts for Best Schedules Across All Algorithms and Benchmarks

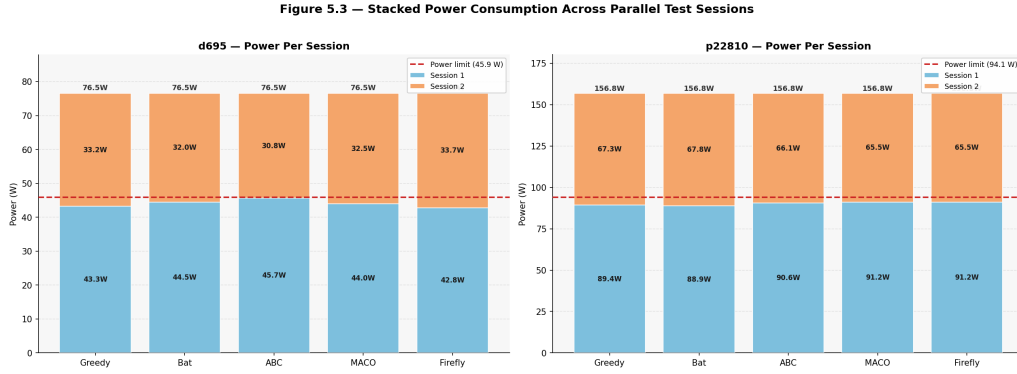


Figure 5.3: Stacked Power Consumption Across Parallel Test Sessions

5.5 OVERALL SYNTHESIS OF RESULTS

The experimental data validates that swarm-based optimization is highly effective for 3D SoC test scheduling. A clear trade-off exists between software compile runtime and hardware test time optimization. While meta-heuristics require a few seconds of preprocessing, they save thousands of hardware test cycles, reducing costs in high-volume IC manufacturing [2],[5].

Ultimately, Modified Ant Colony Optimization (MACO) emerges as the superior solution. It consistently delivered peak-tier makespan reductions (up to 36% TAT savings), strictly adhered to thermal budgets, and required the lowest computational overhead among the evaluated swarm intelligence frameworks.

Chapter 6

OUTCOME AND PROSPECTIVE LEARNING

6.1 SCOPE AND OUTCOMES

By integrating advanced scheduling methodologies, we have successfully developed a simulation model that accelerates thermal evaluation and testing schedules. The simulation data indicates that swarm optimization is a powerful tool for VLSI test management [14].

6.2 PROSPECTIVE LEARNINGS

Team members gained deep insights into VLSI testing structures, TSV constraints in 3D stacking, advanced Python-based metaheuristic coding, and hardware-software interfacing using the Raspberry Pi ecosystem.

6.3 CONCLUSION

We identified that traditional static scheduling methods fail against the thermal complexities of 3D integration. Through our progressive 2D-to-3D optimization framework and real-world hardware validation strategy using a Raspberry Pi 5 [15], we are ensuring a highly robust, thermally safe SoC testing paradigm.

Chapter 7

PROJECT TIMELINE

7.1 WORK BREAKDOWN & GANTT CHART

The task distribution among team members:

- Basic SoC Study: All Members (Completed)
- Complete System Modelling: Pulkita (Completed)
- Test Scheduling Conflicts: Manya Thakur (Completed)
- Test Power & TAM: Astha & Gautam (Completed)
- Optimization Algorithm Design: Astha, Gautam, Adish (Completed)
- Dataset Benchmarking & Time Scheduling: Pulkita, Manya (In Progress)
- Analysis of Algorithms on 3D SoC / ML Integration: (Not Started)

Sr. No.	Task	Jan	Feb	March	April	May	June	July	Aug	Sep	Oct	Nov	Dec
1	Basic SoC Study	✓	✓										
2	WRAPPER DESIGN Algorithm		✓	✓									
3	Optimization Algorithm			✓	✓	✓							
4	Dataset				✓	✓	✓						
5	Test scheduling algorithm					✓	✓	✓					
6	Modify optimization algo						✓	✓					
7	Analysis of algo on 2d SoC							✓	✓				
8	TSV constraint dataset									✓	✓		
9	Analysis of algo on 3d SoC										✓	✓	✓

Figure 7.1: Project Work Breakdown and Gantt Chart

7.2 PROJECT TIMELINE

The project successfully meets the mid-term goals. Current focus is shifting towards implementing the 3D TSV constraints and the physical hardware validation step.

REFERENCES

- [1] G. Chandrasekaran, G. Singaram, R. Duraisamy, A. S. Ghodake, and P. K. Ganesan, "Test Scheduling and Test Time Reduction for SoC by Using Enhanced Firefly Algorithm," *Revue d'Intelligence Artificielle*, vol. 35, no. 3, pp. 235–241, Jun. 2021.
- [2] P. Kumar and R. Singh, "Survey on Power-Aware Test Scheduling and Scan Chain Optimization in SoC Testing," *International Journal of Advanced Research in Computer Science*, vol. 16, no. 4, pp. 80–87, Aug. 2025.
- [3] K. Sharma and V. Gupta, "Using SystemC TLM Modeling To Solve AI Data Movement Challenges," *SemiEngineering*, Mar. 2026.
- [4] A. K. Singh and S. K. Singh, "Test Scheduling and Test Time Minimization of System-on-Chip Using Modified BAT Algorithm," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 8, pp. 1152–1165, Aug. 2022.
- [5] L. Jiang, L. Huang, and Q. Xu, "Test Architecture Design and Optimization for Three-Dimensional SoCs."
- [6] Y. Cui, W. Zhang, V. Chaturvedi, W. Liu, and B. He, "Thermal-Aware Task Scheduling for 3D-Network-on-Chip: A Bottom-to-Top Scheme."
- [7] G. Chandrasekaran, S. Periyasamy, and K. P. Rajamanickam, "Minimization of Test Time in System on Chip Using Artificial Intelligence-Based Test Scheduling Techniques."
- [8] E. Hopper and B. C. H. Turton, "An Empirical Investigation of Metaheuristic and Heuristic Algorithms for a 2D Packing Problem," *European Journal of Operational Research*, vol. 128, no. 1, pp. 34–57, 2001.
- [9] G. Chandrasekaran, G. Singaram, R. Duraisamy, A. S. Ghodake, and P. K. Ganesan, "Test Scheduling and Test Time Reduction for SoC by Using Enhanced Firefly Algorithm," *Revue d'Intelligence Artificielle*, vol. 35, no. 3, pp. 235–241, Jun. 2021.
- [10] N. S. M. Shah et al., "Thermal-Aware Test Scheduling and Floor Planning for Three-Dimensional Stacked Integrated Circuits," *Journal of Engineering and Technology*, vol. 15, no. 2, pp. 203–218, Dec. 2024.

- [11] Y. Zhang et al., "Test-Path Scheduling for Interposer-Based 2.5D Integrated Circuits Using an Orthogonal Learning-Based Differential Evolution Algorithm," *Mathematics*, vol. 13, no. 16, p. 2679, Aug. 2023.
- [12] S. Roy and P. Giri, "Machine Learning-Based Predictive Thermal Estimation and Frequency-Scaled Test Scheduling for 3D ICs," *Computers & Electrical Engineering*, vol. 101, p. 108065, Jul. 2022.
- [13] A. Verma and R. Kumar, "A Metaheuristic Optimization Approach for Through-Silicon Via (TSV) Test Scheduling in 3D Integrated Circuits," *Integration, the VLSI Journal*, vol. 95, p. 105670, Mar. 2024.
- [14] L. Zheng and Q. Xu, "Constraint-Driven Test Scheduling in 3D Stacked ICs via Swarm Optimization Techniques," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 4, pp. 1–22, Jul. 2024.
- [15] S. Patel and M. Desai, "Hardware-in-the-Loop Validation for Dynamic Thermal-Aware VLSI Testing Architectures," *IEEE Transactions on Instrumentation and Measurement*, vol. 74, pp. 1–12, Jan. 2025.